WEEK 1

Name: Dhaksin Kaarthick S.U

Roll no:CH.SC.U4CSE24115

Program 1:

Code:

```c
// CH.SC.U4CSE24115
#include <stdio.h>
int sumofn(int n)
{
    int sum = 0;
    for (int i = 1; i <= n; i++)
    {
        sum += i;
    }
    return sum;
}
int main()
{
    int n;
    printf("Enter number of terms:\n");
    scanf("%d", &n);
    printf("The sum  %d  natural number is: %d\n", n, sumofn(n));
    return 0;
}
```

Output:

```
Enter number of terms:
5
The sum  5  natural number is: 15
```

Space-Complexity Justification:

Total Variables: Sum,n,return sum=3

3*4=12 bytes space

The algorithm uses only a fixed number of variables (sum, n and return n), and no additional memory is allocated based on the input size. Therefore, the memory usage remains constant.

Space Complexity = O(1)

Program 2:

Code:

```c
//CH.SC.U4CSE24115
#include <stdio.h>
int main(){
int n,sum=0;
printf("Enter number of terms: ");
scanf("%d",&n);
for(int i=1;i<=n;i++){
sum+=(i*i);
}
printf("The sum of  square of %d natural numbers is:%d\n",n,sum);
return 0;
}
```

Output:

```
Enter number of terms: 5
The sum of  square of 5 natural numbers is:55
```

Space-Complexity Justification:

Total Variables: Sum,n,=2

2*4=8 bytes space

The algorithm uses only a fixed number of variables (sum and n), and no additional memory is allocated based on the input size. Therefore, the memory usage remains constant.

Space Complexity = O(1)

Program 3:

Code:

```c
//CH.SC.U4CSE24115
#include <stdio.h>
int main(){
int n,sum=0;
printf("Enter number of terms: ");
scanf("%d",&n);
for(int i=1;i<=n;i++){
sum+=(i*i*i);
}
printf("The sum of  cubes of %d natural numbers is:%d\n",n,sum);
return 0;
}
```

Output:

```
Enter number of terms: 5
The sum of  cubes of 5 natural numbers is:225
```

Space-Complexity Justification:

Total Variables: Sum,n,=2

2*4=8 bytes space

The algorithm uses only a fixed number of variables (sum and n), and no additional memory is allocated based on the input size. Therefore, the memory usage remains constant.

Space Complexity = O(1)

Program 4:

Code:

```c
//CH.SC.U4CSE24115
#include <stdio.h>
int fac(int n){
if (n==1){
return 1;
}
else {
return n*fac(n-1);
}

}

int main(){
int n;
printf("Enter no of terms:");
scanf("%d",&n);
printf("The factorial of %d is :%d\n " ,n,fac(n));
return 0;
}
```

Output:

```
Enter no of terms:5
The factorial of 5 is :120
```

Space-Complexity Justification:

Total Variables: n and there is recursive call in program.

4+4*n bytes space

Ignore the constant

The space complexity of the recursive fac function is O(n)

Program 5:

Code:

```c
//CH.SC.U4CSE24115
#include <stdio.h>
int main(){
int mat[3][3]={{1,2,3},{4,5,6},{7,8,9}};

for(int i=0;i<3;i++){
for(int j=0;j<3;j++){
printf("%d ",mat[j][i]);
}
printf("\n");
}
}
```

Output:

```
1 4 7
2 5 8
3 6 9
```

Space-Complexity Justification:

Total Variables: no variables but consider n=row and m=column

Then space will be m*n*4 bytes here 3*3*4=36 bytes

The space complexity of the function is O(m*n) here O(1)

Program 6:

Code:

```c
//CH.SC.U4CSE24115
#include <stdio.h>
int main(){
int n;
printf("enter no of terms: ");
scanf("%d ",&n);
int first=0,second=1,next=0;
for(int i=1;i<=n;i++){
printf("%d ",first);
next=first+second;
first=second;
second=next;
}
return 0;
```

Output:

```
enter no of terms: 5
0 1 1 2 3
```

Space-Complexity Justification:

Total Variables: first second next and n=4

Then space will be 4*4 =16bytes

The space complexity is O(1)